ESP32 QEMU Bare-Metal Walkthrough

A README-style guide for setting up WSL2, ESP-IDF, the Xtensa toolchain, and QEMU, then building and running the final ESP32 QEMU lab from a clean workspace.

This version is written so a supervisor or classmate can follow the steps without the chat log that produced them. It includes the installation steps, the working project structure, the final source files, and the proof that the program ran in QEMU.

1. Overview

The project runs an ESP32 application in QEMU and drives GPIO 2 directly through memory-mapped registers. The serial output proves that the code is executing, and the blink pattern is the same one used in the working terminal session: two quick blinks, two long blinks, then a steady on/off hold.

The important lesson from the build session is that the fully bare-metal linker path was fragile, while the ESP-IDF minimal project worked correctly and still allowed low-level register access.

2. Install WSL2, ESP-IDF, the Xtensa toolchain, and QEMU

Start with WSL2 on Windows, then install the Linux packages inside Ubuntu. After that, install ESP-IDF and let it fetch the Xtensa toolchain and the QEMU support tools.

Installation commands

```
# Windows PowerShell (run as Administrator)
wsl --install
wsl --update
wsl --set-default-version 2

# In Ubuntu inside WSL2
sudo apt update
sudo apt upgrade -y

sudo apt install -y git wget flex bison gperf cmake ninja-build ccache \
    libffi-dev libssl-dev dfu-util python3 python3-pip python3-venv \
    unzip xz-utils file make

mkdir -p ~/esp
cd ~/esp
git clone -b v5.2 --recursive https://github.com/espressif/esp-idf.git
cd ~/esp/esp-idf
./install.sh esp32

. $HOME/esp/esp-idf/export.sh

xtensa-esp-elf-gcc --version
idf.py --version
which qemu-system-xtensa
```

If the compiler command is missing, the environment has not been exported yet. In the successful session the compiler name was xtensa-esp-elf-gcc.

3. Make the environment permanent

Add the ESP-IDF export commands to your shell startup file so every new WSL terminal is ready immediately.

Permanent PATH setup

```
echo 'export IDF_PATH="$HOME/esp/esp-idf"' >> ~/.bashrc
echo '. $IDF_PATH/export.sh' >> ~/.bashrc

source ~/.bashrc

echo "$IDF_PATH"
xtensa-esp-elf-gcc --version
idf.py --version
```

4. Create the project workspace

Create a working directory for the lab and then make the folder structure. The early scratch layout used src, include, and build. The final working version moved into the standard ESP-IDF minimal project structure, but this command set is still useful for a clean workspace.

Workspace commands

```
mkdir -p ~/esp32-qemu-lab
cd ~/esp32-qemu-lab

export IDF_PATH="$HOME/esp/esp-idf"
. $IDF_PATH/export.sh

mkdir -p src include build
```

5. Working solution: use the ESP-IDF minimal project

ESP-IDF handles startup, linking, and memory layout correctly. We keep the project logic bare-metal in style by reading and writing registers directly, but the project itself uses ESP-IDF's normal project skeleton.

The root folder contains CMakeLists.txt and the main folder contains the application source files. The app entry point is app_main(), which is the function ESP-IDF calls after boot.

Project layout commands

```
cd ~
rm -rf ~/esp32-qemu-lab
mkdir -p ~/esp32-qemu-lab
cd ~/esp32-qemu-lab

cp -r $IDF_PATH/examples/get-started/sample_project/* .

idf.py set-target esp32
idf.py build

# Your project folder should now look similar to:
# CMakeLists.txt  README.md  build  main  sdkconfig  sdkconfig.old
```

6. Final source code

Create these files inside the main folder. The code below is the final version used in the successful build and QEMU run.

6.1 main/esp32_regs.h

esp32_regs.h

```
#ifndef ESP32_REGS_H
#define ESP32_REGS_H

#include <stdint.h>

/* GPIO Matrix registers */
#define GPIO_ENABLE_REG     (*(volatile uint32_t *)0x3FF44020)
#define GPIO_ENABLE_W1TS    (*(volatile uint32_t *)0x3FF44024)
#define GPIO_OUT_REG        (*(volatile uint32_t *)0x3FF44004)
#define GPIO_OUT_W1TS_REG   (*(volatile uint32_t *)0x3FF44008)
#define GPIO_OUT_W1TC_REG   (*(volatile uint32_t *)0x3FF4400C)
#define GPIO_IN_REG         (*(volatile uint32_t *)0x3FF4403C)

/* UART0 registers */
#define UART0_BASE          0x3FF40000
#define UART_FIFO_REG       (*(volatile uint32_t *)(UART0_BASE + 0x00))
#define UART_STATUS_REG     (*(volatile uint32_t *)(UART0_BASE + 0x1C))
#define UART_CLKDIV_REG     (*(volatile uint32_t *)(UART0_BASE + 0x14))
#define UART_CONF0_REG      (*(volatile uint32_t *)(UART0_BASE + 0x20))

#define GPIO2_BIT           (1U << 2)

#endif /* ESP32_REGS_H */
```

6.2 main/gpio.c

gpio.c

```
#include "esp32_regs.h"

void gpio_init_output(int pin) {
    volatile uint32_t *mux = (volatile uint32_t *)(0x3FF49000 + pin * 4);
    *mux = (*mux & ~0x13) | (2 << 12);
    GPIO_ENABLE_W1TS = (1U << pin);
}

void gpio_set(int pin) {
    GPIO_OUT_W1TS_REG = (1U << pin);
}

void gpio_clear(int pin) {
    GPIO_OUT_W1TC_REG = (1U << pin);
}
```

6.3 main/uart.c

uart.c

```
#include "esp32_regs.h"

void uart_init(void) {
    UART_CLKDIV_REG = (80000000 / 115200) << 4;
    UART_CONF0_REG = 0x00000003;
}

void uart_putc(char c) {
    while (UART_STATUS_REG & 0x00400000);
    UART_FIFO_REG = c;
}

void uart_puts(const char *s) {
    while (*s) {
        uart_putc(*s++);
    }
}
```

6.4 main/main.c

main.c

```
#include <stdint.h>
#include "esp32_regs.h"

void gpio_init_output(int pin);
void gpio_set(int pin);
void gpio_clear(int pin);
void uart_init(void);
void uart_puts(const char *s);

#define QUICK_MS    200
#define LONG_MS     800
#define HOLD_MS     2000

void delay_ms(uint32_t ms) {
    uint32_t cycles = ms * 240000;
    while (cycles--) {
        __asm__ volatile ("nop");
    }
}

void blink_quick(void) {
    gpio_set(2);
    delay_ms(QUICK_MS);
    gpio_clear(2);
    delay_ms(QUICK_MS);
}

void blink_long(void) {
    gpio_set(2);
    delay_ms(LONG_MS);
    gpio_clear(2);
    delay_ms(LONG_MS);
}

void hold_steady(void) {
    gpio_set(2);
    delay_ms(HOLD_MS);
    gpio_clear(2);
    delay_ms(HOLD_MS);
}

void app_main(void) {
    uart_init();
    gpio_init_output(2);

    uart_puts("\r\n=== ESP32 QEMU Bare-Metal ===\r\n");
    uart_puts("Pattern: 2 quick -> 2 long -> 2s on -> 2s off\r\n");
    uart_puts("================================\r\n\r\n");

    while (1) {
        uart_puts("CYCLE START\r\n");

        blink_quick();
        blink_quick();
        uart_puts("  2 quick done\r\n");

        blink_long();
        blink_long();
        uart_puts("  2 long done\r\n");

        hold_steady();
        uart_puts("  hold done\r\n\r\n");
    }
}
```

6.5 Root CMakeLists.txt and main/CMakeLists.txt

CMake files

```
# Root CMakeLists.txt
cmake_minimum_required(VERSION 3.16)
include($ENV{IDF_PATH}/tools/cmake/project.cmake)
project(esp32_qemu_lab)

# main/CMakeLists.txt
idf_component_register(SRCS "main.c" "gpio.c" "uart.c"
                       INCLUDE_DIRS ".")
```

7. Build the project

Once the files are in place, build with ESP-IDF. The build should generate the bootloader, partition table, and application binaries inside the build folder.

Build commands

```
cd ~/esp32-qemu-lab

idf.py set-target esp32
idf.py build
```

8. Create the QEMU flash image and run it

After the build succeeds, merge the binaries into a single flash image with esptool.py, pad it to 4 MB, and run QEMU. This is the command sequence that produced the working terminal proof.

Flash image and QEMU commands

```
cd ~/esp32-qemu-lab

esptool.py --chip esp32 merge_bin \
    -o build/qemu_flash.bin \
    --flash_mode dio \
    --flash_size 4MB \
    --flash_freq 40m \
    0x1000 build/bootloader/bootloader.bin \
    0x8000 build/partition_table/partition-table.bin \
    0x10000 build/main.bin

truncate -s 4M build/qemu_flash.bin

qemu-system-xtensa -nographic \
    -machine esp32 \
    -drive file=build/qemu_flash.bin,if=mtd,format=raw \
    -serial mon:stdio \
    -global driver=timer.esp32.timg,property=wdt_disable,value=true
```

To stop QEMU, press Ctrl+A then X. If you need the monitor, press Ctrl+A then C and type quit.

9. Proof that it works

The final QEMU run printed the ESP32 boot messages and then the application output below. This is the visible proof that the build, flash image, serial output, and application loop all worked together.

Working terminal output

```
Adding SPI flash device
ets Jul 29 2019 12:21:46

rst:0x1 (POWERON_RESET),boot:0x12 (SPI_FAST_FLASH_BOOT)

=== ESP32 QEMU Bare-Metal ===
Pattern: 2 quick -> 2 long -> 2s on -> 2s off
================================

CYCLE START
  2 quick done
  2 long done
  hold done
```

10. Project root after the build

After a successful build, the project root should look similar to the listing below.

Directory listing

```
hushfire@DESKTOP-1ISV5KK:~/esp32-qemu-lab$ ls
CMakeLists.txt  README.md  build  main  sdkconfig  sdkconfig.old
```

11. Troubleshooting notes

Problem	What to check
xtensa-esp-elf-gcc not found	Run . $HOME/esp/esp-idf/export.sh and confirm ~/.bashrc sources it.

Problem	What to check
idf.py not found	The ESP-IDF environment was not exported or IDF_PATH is wrong.
qemu-system-xtensa missing	Install the QEMU tool through ESP-IDF tools and re-export the environment.
Nothing prints in QEMU	Confirm the flash image was built with esptool.py merge_bin and that app_main() is used.
QEMU monitor and serial conflict	Use -serial mon:stdio or add -monitor none.

That is the complete working walkthrough. The important part is not the first bare-metal linker attempt; it is the final ESP-IDF-based version that compiles cleanly, runs in QEMU, and prints the expected pattern.